
Simulation des spezifischen Verhaltens von Bienen bei der Nahrungssuche als Multiagentensystem

Version 1.0

Autoren

Jana Gerhard

Lukas Klug

Felix Müller

Nico Schulze-Bilk

Fakultät für Mathematik und Informatik

Praktikum Multiagentensysteme

Inhalt

1.	Einleitung	4
1.1.	Was ist ein MAS?	4
1.2.	Bienen in der Natur.....	4
1.2.1	Struktur eines Bienenvolks	5
1.2.2.	Wie verhalten sich Bienen bei der Futtersuche	5
2.	Analyse Bienenalgorithmen	8
2.1.	Recherche von bestehenden Algorithmen	8
2.1.1.	Bees Algorithm	8
2.1.2.	ABC Algorithmus.....	9
2.2.	Notwendige Anpassungen	10
2.3.	Modelanforderungen	11
3.	Modell	12
3.1.	Verwendungszweck	12
3.2.	Entwicklungsumgebung und Simulationsbibliothek	12
3.3.	Input Parameter und Input Dokumente	13
3.4.	Entitäten und Zustandsvariablen.....	14
3.5.	Prozessübersicht	17
3.6.	Grundprinzipien.....	20
3.6.1.	Tanzwahrscheinlichkeit	20
3.6.2.	Ermittlung der maximalen Sucherinnen	21
3.6.3.	Tanzfläche und Schwänzeltanz	22
3.6.4.	Berechnung der Tanzzeit.....	22
3.6.5.	Rekrutierung der Zuschauerinnen.....	23
3.6.7	Entdecken und Ernten von Nahrungsquellen.....	23
3.7	Initialisierung	24
3.7.1	Initialisierung der Umgebung	24
3.7.2	Initialisierung der Community.....	24
4	Simulation	25
4.1	Parameteranpassungen und Annahmen	26
4.2	Beobachtungen	26
4.2.1.	Priorisierung	26
4.2.2.	Algorithmen	28
4.2.3.	Kapazität	29
4.3.	Zusammenfassung	30
5	Fazit	32
6.	Literatur	33

1. Einleitung

Im Rahmen dieses Praktikums haben wir ein Multiagentensystem (MAS) entwickelt, das die Futtersuche von Bienen in einer virtuellen Landschaft simuliert. Die Herausforderung bei der Entwicklung dieses MAS bestand darin, effektive Mechanismen für die Kommunikation, Koordination und Kooperation zwischen den Agenten zu implementieren.

Das übergeordnete Ziel unseres Bienensystems ist es, die Komplexität einer effektiven Futtersuche in kleinere, handhabbare Unteraufgaben zu unterteilen und diese Aufgaben an die Agenten (Bienen) zu delegieren. Durch die Entscheidungsfähigkeit der einzelnen Agenten soll eine flexible Futtersuche implementiert werden, die sich an verschiedene und wechselnde Gegebenheiten anpassen kann.

1.1. Was ist ein MAS?

Ein Multiagentensystem kann als lose gekoppeltes Netzwerk von Agenten definiert werden, die zusammen Probleme lösen, die über die individuelle Fähigkeiten oder das Wissen den einzelnen Agenten hinausgeht (Durfee & Lesser, 1989).

Diese Agenten sind autonom und können sich voneinander unterscheiden (heterogen). Die Merkmale von MAS sind, dass jeder Agent über unvollständige Informationen oder Fähigkeiten verfügt und somit nur eine begrenzte „Sichtweite“ hat. Bei Multiagentensystemen gibt es keine globale Kontrolleinheit, die Daten sind somit dezentralisiert und die Berechnungen erfolgen asynchron (K. P. Sycara, 1998).

Multiagentensysteme bieten einige Eigenschaften, die sie zu interessanten Werkzeugen in der Informatik machen

Zum einen können MAS Probleme lösen, die für einen zentralisierten Agenten aufgrund von Ressourcenbeschränkungen zu groß sind. Zum anderen können MAS Informationsquellen nutzen, die räumlich verteilt sind. Beispiele für solche Bereiche umfassen Sensornetzwerke (Corkill & Lesser, o. J.) und die Internetsuche (K. Sycara et al., 1996).

Neben weiteren Anwendungsgebieten eignen sich MAS zudem sehr gut zur Lösung von Problemen die aufgrund ihrer inhärenten Natur als eine Gesellschaft von autonomen, interagierenden Komponenten (Agenten) betrachtet werden können. Als Anwendungsbeispiele können hier die Luftverkehrskontrolle, Prozessleitsysteme aber auch ökologische und soziale Systeme genannt werden. Als ein solcher ökologischer Anwendungsfall kann auch das hier entwickelte Bienenmodell betrachtet werden, welches die Modellierung und die Analyse des Verhaltens der einzelnen Tiere und deren Auswirkung auf das Gesamtsystem zulässt.

1.2. Bienen in der Natur

Da das im Zuge dieses Praktikums entwickelte Model sich in erster Linie an dem Aufbau und dem Verhalten eines in der Natur vorkommenden Bienenvolks orientieren soll, hält der folgende Abschnitt einige grundsätzlichen Informationen über die Honigbiene *Apis mellifera* bereit. Als primärer Aspekt des Models, wird insbesondere das Verhalten der Bienen bei der Futtersuche näher betrachtet.

1.2.1 Struktur eines Bienenvolks

Ein gesundes Bienenvolk besteht aus ca. 40.000 – 70.000 adulten Bienen. Diese teilen sich in 3 Obergruppen auf; Königinnen, Arbeiterbienen und Drohnen (Abbildung 1.1).

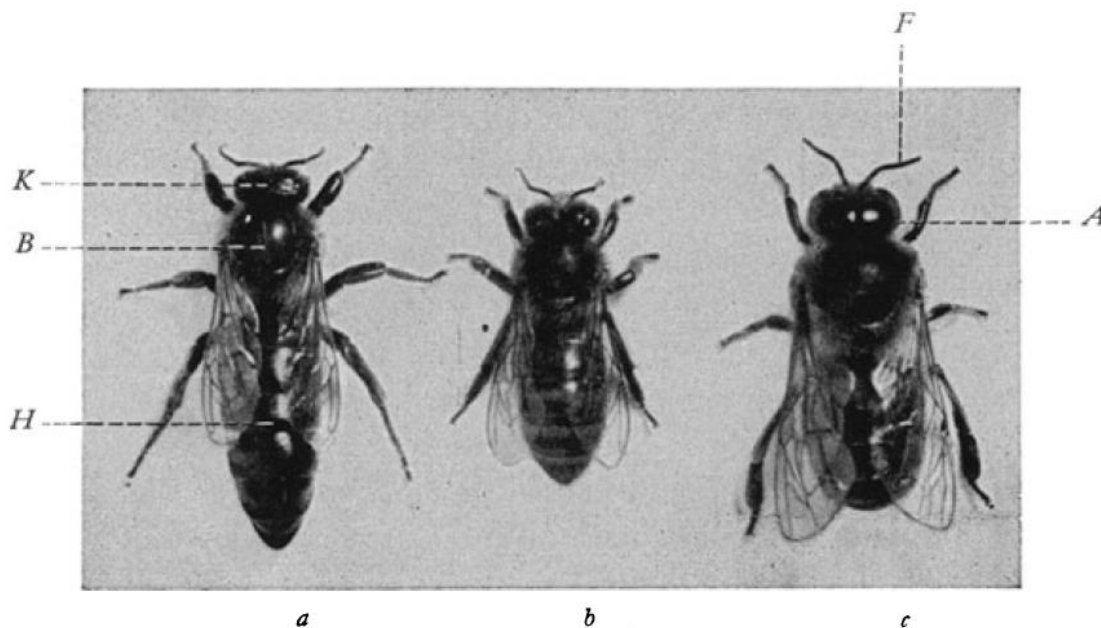


Abbildung 1.1: a Königin, b Arbeiterin, c Drohne. K Kopf, B Brust, H Hinterleib, A Auge, F Fühler (V. Frisch, 1964)

In jedem Bienenvolk gibt es lediglich eine einzige Königin, diese ist das einzige vollentwickelte Weibchen und zur Fortpflanzung fähig. Im Frühjahr kann eine leistungsfähige Königin in 24 Stunden etwa 1500 Eier legen. Die Lebensspanne der männlichen Tiere (Drohnen) begrenzt sich auf das Frühjahr, während dieser Zeit begatten sie die Königin auf deren „Hochzeitsflügen“. Nach der Begattung hat die Königin in einem Samenbehälter am Hinterleib die männlichen Keimzellen gespeichert, die mehrere Jahre lebendig und gebrauchsfähig bleiben.

Alle anderen Tiere sind Arbeitsbienen, bei diesen handelt es sich um fortpflanzungsunfähigen Weibchen (V. Frisch, 1964). Die Arbeiterinnen betreiben eine strukturierte Arbeitsteilung, bei der einzelne Individuen in verschiedenen Altersphasen unterschiedliche Aufgaben ausführen (altersabhängige Arbeitsteilung). Die Arbeiterbienen übernehmen dabei die Brutpflege, den Wabenbau, die Verteidigung des Stocks sowie die Futterspeicherung und **Futtersuche** (Wehner & Gehring, 2013).

Da wir uns bei unseren Simulationen auf die Futtersuche der Bienen konzentriert habe, ist diese in Kapitel 1.2.2 näher beschrieben (Abou-Shaara, 2014).

1.2.2. Wie verhalten sich Bienen bei der Futtersuche

Die Sammlerbienen (zuständig für die Futtersuche) lassen sich in drei Kategorien einteilen. Sucher sowie Sammler, die nach der besten Nahrungsquelle suchen und Zuschauer Bienen. Die Zuschauer warten im Bienenstock, bis die Kundschafterbienen zurückkehren und ihnen Informationen über die gefunden Nahrungsquellen mittels des

Schwänzeltanzes weitergeben. Die Zuschauer Bienen machen hierbei im Allgemeinen 40-90 % der Gesamtpopulation der Sammlerinnen aus (Abou-Shaara, 2014). Die Honigbienen beginnen im allgemeinen mit der Futtersuche am frühen Morgen und beenden diese erst wieder am Abend. Während dieser Zeit legen die Bienen, in Abhängigkeit der Jahreszeit und des Nahrungsangebots im Mittel zwischen ca. 600m und 3000m zurück (Abou-Shaara, 2014).

Sammlerinnen bevorzugen manchmal eine Nahrungsquelle vor einer anderen, des weiteren kann die spezifische Position einer Blüte gegenüber einer anderen präferiert werden. (Sushil et al., 2013) fanden z. B. heraus, dass die meisten Sammlerinnen Brokkoli gefolgt von Kohlrabi bevorzugen und Chinakohl als letztes anfliegen.

Es gibt verschiedene Faktoren, die das Suchverhalten von Bienen beeinflussen können. Betrachtet man verschiedene Umweltfaktoren kann sowohl die Temperatur, die Luftfeuchtigkeit und das Wetter Auswirkungen auf das Suchverhalten der Bienen haben (Abou-Shaara, 2014).

Schwänzeltanz

Bei sozialen Insekten dominiert mit der Kommunikation mittels Pheromone der chemische Informationskanal (z.B. Ameisen). „Darüber hinaus findet sich bei der Honigbienen (*Apis mellifera*) aber auch ein **abstraktes Signalsystem**, mit dem sich Koloniemitglieder über Richtung und Entfernung zu ergiebigen Futterquellen verständigen“ (Wehner & Gehring, 2013). Mit dem Schwänzeltanz definiert eine erfolgreich heimgekehrte Biene die Richtung zur Futterquelle als Winkel zwischen Sonnenazimut und Futterquelle. Die Entfernung zur Futterquelle wird über die Dauer des Tanzes kommuniziert. Je länger die Schwänzelpause, desto weiter ist die Futterquelle entfernt (Abbildung 1.2) (Wehner & Gehring, 2013)

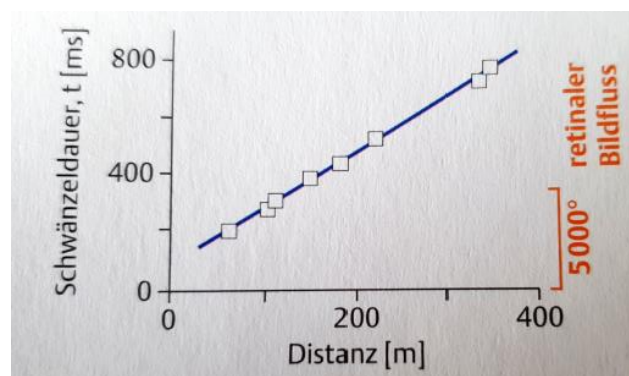


Abbildung 1.2: Schwänzeldauer als Funktion der Flugdistanz (Wehner & Gehring, 2013).

Flugverhalten

Allen Bienenarten gemeinsam ist ihre Fähigkeit zu fliegen: Ob bei der Suche nach Nahrungsquellen, bei der Navigation zurück in den Bienenstock oder beim Paarungsflug, viele wichtige Verhaltensweisen werden fliegend ausgeführt. Die Vermeidung von Kollisionen ist für einen effizienten Flug unerlässlich. Für Bienen ist die Kollisionsvermeidung jedoch nicht trivial, da das visuelle System der Bienen

nicht in der Lage ist, die räumliche Tiefe durch stereoskopisches Sehen exakt zu ermitteln. Die effizienteste Lösung für eine Biene, Kollisionen zu vermeiden besteht somit darin, den Abstand zu nahen Hindernissen zu maximieren (Baird et al., 2020).

2. Analyse Bienenalgorithmen

Das natürliche Vorbild für Bienenalgorithmen ist das Verhalten der westlichen Honigbienen bei der Futtersuche (Hollstein, 2023).

Beim Suchen nach der besten Futterquelle imitieren Bienenalgorithmen die Aufgabenverteilung zwischen Späherinnen, Sammlerinnen und Zuschauerinnen im Bienenstock. Ähnlich wie im natürlichen Vorbild übernehmen die Sammlerinnen die lokale Suche nach optimalen Lösungen im Suchraum, während die Späherinnen für die globale Suche zuständig sind.

2.1. Recherche von bestehenden Algorithmen

Mit Hilfe von Bienenalgorithmen können sowohl kombinatorische als auch kontinuierliche Optimierungsprobleme gelöst werden. Die ersten Bienenalgorithmen wurden 2005 von Pham et al. und Karaboga entwickelt, und werden in diesem Kapitel näher erläutert werden.

Der Hauptunterschied zwischen dem BA-Algorithmus von Pham und dem ABC-Algorithmus von Karabo liegt in der jeweiligen Strategie der lokalen Suche.

2.1.1. Bees Algorithm

Der Bienenalgorithmus (Bees Algorithm) simuliert das Verhalten von Bienen bei der Suche nach Futterquellen, um kontinuierliche Optimierungsprobleme zu lösen. Dabei wird jede potenzielle Lösung als Lokation einer Futterquelle aufgefasst. Die Qualität der Futterquelle, kann dabei als Fitnessfunktion der Lösung betrachtet und die Umgebung eines Punktes im Lösungsraum kann als Blühlandschaft aufgefasst werden (Hollstein, 2023).

Zu Beginn des Algorithmus werden n Späherinnen zufällig im Suchraum auf n Futterquellen (Lösungen) verteilt und nach ihrer Fitness bewertet (globale Suche). Die besten m Futterquellen werden für die lokale Suche ausgewählt. Unter diesen m Futterquellen werden den Umgebungen der e elitären Stellen jeweils nep Sammlerinnen zugewiesen. Die restlichen besten $m - e$ Futterquellen erhalten nsp Sammlerinnen für die lokale Suche, mit $nsp < nep$.

Im Anschluss wird für jede der m Umgebungen der höchste Fitnesswert ausgewählt, dessen Biene daraufhin zur neuen Späherin wird. Am Ende der Iteration werden die übrigen Späherinnen zufällig im Suchraum verteilt, um neue potenzielle Lösungen zu finden (Hollstein, 2023).

Abbildung 2.1 stellt den grundsätzlichen Ablauf des Bees Algorithm graphisch da.

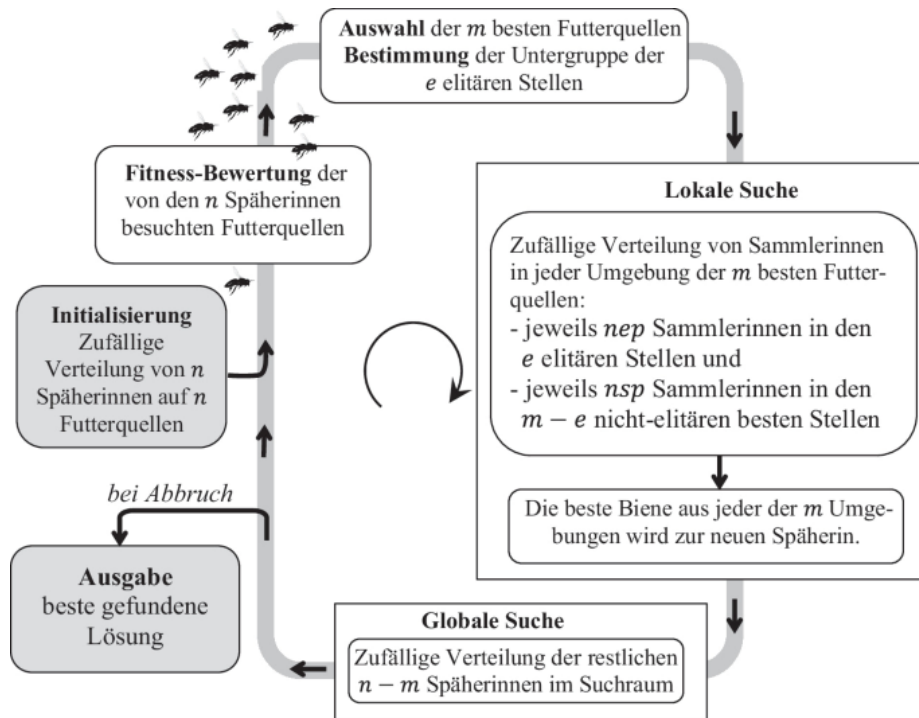


Abbildung 2.1: Ablaufdiagramm des Bee-Algorithm nach (Hollstein, 2023)

2.1.2.ABC Algorithmus

Der ABC-Algorithmus (Artificial Bee Colony Algorithm) wurde von Karaboga zur Lösung kontinuierlicher Optimierungsprobleme eingeführt (Hollstein, 2023). Der Hauptunterschied zwischen dem ABC-Algorithmus und dem BA-Algorithmus liegt in der unterschiedlichen Strategie der lokalen Suche. Wie bei der BA-Methode repräsentiert die Position der Futterquelle eine potenzielle Lösung des Optimierungsproblems und die Nektarmenge der Futterquelle die Fitness. Beim ABC-Verfahren besteht die Population der künstlichen Bienen aus drei Gruppen: den Sammlerinnen, den Zuschauerinnen und den Späherinnen. Die Anzahl der Sammlerinnen und der Zuschauerinnen ist jeweils SN . Die Sammlerinnen identifizieren SN Futterquellen (Lösungen) $x_i, i = 1, \dots, SN$, ebenso werden den Zuschauerinnen SN Futterquellen $y_i, i = 1, \dots, SN$, zugewiesen. Bei den Lösungen von x_i und y_i handelt es sich um D-dimensionale Vektoren (Hollstein, 2023).

Beim ABC-Algorithmus gliedert sich die lokale Suche somit in eine weitere Phase auf, die Phase der Zuschauerinnen. Diese Phase schließt sich an die lokale Suche der Sammlerinnen an und beginnt damit, dass die Sammlerinnen, die Zuschauerinnen über die Güte der einzelnen Futterquellen aus der lokalen Suche informieren. Die Zuschauerinnen treffen die Entscheidung zu welcher Futterquelle x_m sie rekrutiert werden anhand der Roulette-Methode, die zu einer höheren Rekrutierungswahrscheinlichkeit führt, je größer der Fitnesswert der Futterquelle ist (Hollstein, 2023).

Ist für eine Zuschauerin probabilistisch die Futterquelle x_m bestimmt worden, so führt sie, wie die Sammlerinnen eine lokale Suche in der Umgebung von x_m durch. Hat die modifizierte Lösung einen höheren Fitnesswert, so verlässt die Zuschauerin ihre alte Position der Voriteration.

Sammlerinnen, deren Lösungen nach einer vom Anwender vorgegebenen Anzahl von Iterationen nicht verbessert werden können, verlassen ihre Futterquelle, da sie nunmehr als ausgebeutet gilt. In diesem Fall konvertieren diese Sammlerinnen zu Späherinnen und startet eine neue globale Suche (Hollstein, 2023).

2.2. Notwendige Anpassungen

Die beiden vorgestellten Bienenalgorithmen nehmen, wie bereits erwähnt das natürliche Verhalten der Bienen als Vorlage um allgemeinen Optimierungsprobleme zu lösen. Ziel dieser Algorithmen ist es jedoch nicht, dass tatsächlich Suchverhalten von Bienen realistisch abzubilden. Da die möglichst naturnahe Simulation des Suchverhaltens eines Bienenvolkes jedoch Ziel dieser Arbeit ist mussten wir einige Anpassungen an, die bereits bestehenden Algorithmen vornehmen. So entstand im Rahmen dieser Arbeit ein neuer Algorithmus, der Elemente aus beiden bestehenden Algorithmen enthält.

Die Simulation des Nahrungssuchverhalten von Bienen beinhaltet verschiedene Herausforderungen dar. Eine der Hauptschwierigkeiten besteht darin, die komplexen und dynamischen Interaktionen zwischen den Bienen möglichst genau zu modellieren. In der Natur arbeiten Bienen als hoch organisierte soziale Einheiten, die Informationen über Futterquellen effizient austauschen. Diese Kommunikation und Koordination in einem Algorithmus realistisch abzubilden, erfordert eine sorgfältige Recherche der Verhaltensweisen und deren Implementierung.

Ein weiteres Problem ist die Anpassung des Modells an verschiedene Gegebenheiten. Das natürliche Verhalten von Bienen variiert je nach Umgebung und Nahrungsangebot, was bedeutet, dass das Modell flexibel genug sein muss, um unterschiedliche Fragestellungen abdecken zu können.

Die Parametereinstellung ist ebenfalls ein kritischer Aspekt. Die „Performance“ der Bienen hängt stark von der Wahl der Parameter ab, wie zum Beispiel der Anzahl der Späherinnen, Sammlerinnen und Zuschauerinnen, sowie den Regeln für die Suche. Verschiedene Einstellungen können zu suboptimalen Lösungen oder ineffizienten Suchprozessen führen.

Insgesamt erfordert die Simulation des Nahrungssuchverhaltens von Bienen eine sorgfältige Abwägung zwischen biologischer Genauigkeit und algorithmischer Effizienz, um robuste und effektive Lösungen zu bieten. Zusätzlich zu den bereits genannten Herausforderungen stellt die genaue Simulation des biologischen Verhaltens von Bienen eine besondere Schwierigkeit dar. Bienen verfügen über hochentwickelte Mechanismen zur Navigation und Kommunikation, die auf

komplexen sensorischen und kognitiven Prozessen basieren. Diese umfassen zum Beispiel die Fähigkeit, den Sonnenstand zur Orientierung zu nutzen, tanzbasierte Kommunikation zur Übermittlung von Futterquelleninformationen sowie das Erkennen und Unterscheiden von unterschiedlichen Futterquellen. Diese feinen Nuancen des Verhaltens in ein mathematisches Modell zu übertragen, erfordert Kenntnisse der Bienenbiologie und der Fähigkeit, diese Prozesse in algorithmischen Regeln zu kodifizieren.

Insgesamt erfordert die Simulation akkuraten biologischen Verhaltens eine Balance zwischen biologischer Detailtreue und der Praktikabilität des Modells, um sowohl die Effizienz des Optimierungsalgorithmus als auch seine Fähigkeit, realitätsnahe Lösungen zu liefern, sicherzustellen. Im Abschnitt **Fehler! Verweisquelle konnte nicht gefunden werden.** werden wir näher auf die, in diesem Modell implementierten natürlichen Verhaltensweisen der Bienen eingehen und ihre Umsetzung beschreiben.

2.3. Modelanforderungen

Wehner und Gehring (2013) beschreiben das Suchverhalten der Bienen als eine Koordination von arbeitsteiligen erbrachten Leistungen, die dichte Kommunikationsnetze erfordern, damit das Verhaltensrepertoire der Individuen situationsgerecht zum Einsatz kommen kann.

Somit sind Multiagentensysteme besonders gut geeignet, um das Suchverhalten von Bienen zu simulieren. Sie spiegeln die kollektive Intelligenz und die dezentrale Entscheidungsfindung wider, die Bienen in der Natur zeigen. In einem Multiagentensystem agieren verschiedene Agenten autonom und koordiniert, ähnlich wie Bienen in einem Schwarm. Diese Agenten können miteinander kommunizieren, Informationen austauschen und ihre Strategien dynamisch anpassen, um effizient nach Nahrung zu suchen.

Zudem ermöglichen Multiagentensysteme die Modellierung komplexer Verhaltensweisen und Interaktionen in einer simulierten Umgebung, was es uns erlaubt, unterschiedliche Szenarien und Umweltbedingungen zu testen und zu analysieren.

3. Modell

In diesem Abschnitt wird das entwickelte Bienenmodell in Anlehnung an das ODD (Overview, Design concepts, and Details) Protokoll zur Beschreibung von Agenten-basierten Modellen (Grimm et al., 2010) näher beschrieben.

3.1. Verwendungszweck

Das entwickelte Bienenmodell erfüllt den Zweck das Suchverhalten von Bienen in verschiedenen Szenarien abzubilden.

Das Modell erlaubt es, verschiedene Nahrungsquellen in unterschiedlicher Entfernung von einem oder mehreren Bienenstöcken im Suchraum zu platzieren. Je nach Situation werden den Bienen dynamisch unterschiedliche Rollen/Jobs zugewiesen. Die Bienen können dabei die Rolle von Sucherinnen, Sammlerinnen und Zuschauerinnen annehmen.

Die Sucherinnen bewegen sich zufällig durch den Suchraum auf der Suche nach neuen, noch nicht gefundenen Nahrungsquellen. Die Sammlerinnen fliegen schon bekannte Nahrungsquellen an, um an diesen Nektar zu sammeln und in den Stock zu bringen. Sowohl die Sucherinnen als auch die Sammlerinnen teilen die aktuelle Qualität der von ihnen besuchten Nahrungsquelle mit den Zuschauerinnen im Stock. Die Zuschauerinnen entscheiden Aufgrund der Informationsweitergabe, welche Nahrungsquelle sie im Zuge ihres nächsten „Nahrungsflugs“ anfliegen und werden im Zuge dessen zur Sammlerin.

3.2. Entwicklungsumgebung und Simulationsbibliothek

Für die Entwicklung der Simulation wurde die Programmiersprache Python verwendet. Für die Darstellung der Grafiken und der Kollisionserkennung wurde die Python-Programmbibliothek Pygame verwendet.

Die Kollisionserkennung von Entitäten wird mit der Sprite Klasse von Pygame gelöst. Dabei wird eine Kollision zwischen Bienen und Nahrungsquelle, sowie eine Kollision zwischen Bienen und Bienenstock realisiert. Auf eine Kollision zwischen Bienen untereinander wird in der Simulation verzichtet, da sich Bienen in der Realität im 3-dimensionalen Raum bewegen und nicht kollidieren würden.

Um die Simulation im Rahmen der Gruppenarbeit gemeinsam und agil entwickeln und testen zu können, wurde zudem das Versionsverwaltungstool Git und Github verwendet.

3.3. Input Parameter und Input Dokumente

Das Bienen-Modell benötigt einige, vom User vordefinierte Konfigurationsparameter (Tabelle 3-1), die definieren, wie sich die Bienen bei der Nahrungssuche verhalten und welchen Regeln die Bienen unterliegen. Des Weiteren werden mit Hilfe der Konfigurationsparameter die Eigenschaften des virtuellen Suchraums festgelegt.

Tabelle 3-1: Konfigurationsparameter

Name	Typ	Beschreibung
FRAMES_PER_SECOND	Integer	Anzahl der Updatezyklen pro Sekunde
SCREEN_HEIGHT	Integer	Höhe des Fensters
SCREEN_WIDTH	Integer	Breite des Fensters
SIMULATION_WIDTH	Integer	Breite der Simulationsfläche
BEES_SCOUT	Integer	Anzahl Scout Bienen im System zum Start
BEES_ONLOOKER	Integer	Anzahl Onlooker Bienen im System zum Start
MAX_BEES_SCOUT	Integer	Anzahl maximale Scout Bienen im System
MAX_BEES_DANCER	Integer	Anzahl maximale tanzende Bienen
FOOD_COUNT	Integer	Anzahl Futterquellen im System
MAX_SUGAR	Integer	Maximale Anzahl Zuckereinheiten pro Futterquelle
MIN_SUGAR	Integer	Minimale Anzahl an Zuckereinheiten pro Futterquelle
MAX_UNITS	Integer	Maximale Futterkapazität einer Futterquelle
MIN_UNITS	Integer	Minimale Futterkapazität einer Futterquelle
MAX_TIME	Integer	Anzahl Zeit der Simulation
MIN_RANGE_FOOD_TO_HIVE	Integer	Wie weit müssen Futterquellen mindestens vom Bienenstock entfernt sein
MAX_VELOCITY_BEE	Integer	Maximale Geschwindigkeit einer Biene
MIN_VELOCITY_BEE	Integer	Minimale Geschwindigkeit einer Biene
WALKING_SPEED	Float	Faktor für die Gehgeschwindigkeit einer Biene
BEE_VISION	Integer	Sichtradius einer Biene
BEE_MAX_CAPACITY	Integer	Maximale Anzahl Futter das eine Biene tragen kann
REDUCE_SPEED_WHEN_CARRY	Integer	Reduktion der Geschwindigkeit, wenn die Biene Nahrung trägt
MAX_STEP_COUNTER_BEES	Integer	Anzahl Schritte bevor die Biene zurückkehren muss
DANCETIME_PER_UNIT	Integer	Dauer eines Tanzes
DANCEFLOOR_RADIUS	Integer	Radius einer einzelnen Tanzfläche
DANCEFLOOR_CAPACITY	Integer	Futterquelle an der Biene Nahrung sammelt
MIN_DANCE_PROBABILITY	Float	Minimale Wahrscheinlichkeit mit der eine Biene für eine Quelle tanzt
MAX_DANCES_WATCHED	Integer	Anzahl der Tänze die ein Onlooker maximal schaut
MIN_DANCES_WATCHED	Integer	Anzahl der Tänze die ein Onlooker mindestens schaut

Die genaue Bedeutung und Auswirkung der einzelnen Parameter ist im Abschnitt 3.6, unter den aufgeführten Grundprinzipien näher erläutert.

Die Initialisierung der Landschaft kann entweder randomisiert erfolgen oder durch das Einlesen eines Environment-Files. Mit diesem werden die Positionen der Hives und der Nahrungsquellen, sowie die Nahrungsqualität der einzelnen Quellen festgelegt. Auf diese Weise sind vergleichende und auswertbare Simulationen mit verschiedenen Parametereinstellung in einer definierten Landschaft möglich. Eine genauere Beschreibung des Initialisierungsprozesses ist im Abschnitt 3.7 zu finden.

3.4. Entitäten und Zustandsvariablen

Die folgenden Entitäten sind im Bienen-Modell enthalten: Bienen, die aktiv an der Nahrungssuche beteiligt sind (d.h. Sucherinnen, Sammlerinnen und Zuschauerinnen), Landschaftselemente (d. h. Bienenstöcke inkl. Tanzflächen und Nahrungsquellen), die einen geographischen Standort in der Landschaft besitzen und mit denen die Bienen interagieren und die globale Umgebung, die die zu durchsuchende Landschaft repräsentiert. Bienen sind immer fest einem Bienenstock zugeordnet, während Nahrungsquellen ohne Zuordnung zufällig im Raum platziert werden. Eine Übersicht zu den zugehörigen Statusvariablen findet sich in Table 3-2 – 3.5.

Bei den Sucherinnen handelt es sich um Bienen die den gesamten Suchraum randomisiert durchfliegen um neue, noch unbekannte Nahrungsquellen zu finden. Wichtige Informationen einer gefundenen Nahrungsquelle sollen dann von den Sucherinnen an andere Bienen übergeben werden.

Die Sammlerinnen sind einer bestimmten Futterquelle zugeordnet und fliegen diese zur Nahrungsbeschaffung direkt an, bis sie komplett abgeerntet ist.

Bei den Zuschauerinnen handelt es sich um Bienen innerhalb des Bienenstockes die den anderen Bienen beim Schwänzeltanz zusehen und sich in Zuge dessen zum Anflug einer geteilten Nahrungsquelle entscheiden können.

Die Bienenstöcke dienen als Ausgangspunkt, Futterlager und Kommunikationsplattform für ein Bienenvolk. Alle Bienen eines Volkes sammeln das Futter im zugehörigen Bienenstock. Bienen, die kein Futter gefunden haben, kehren in den Bienenstock zurück und kommunizieren dort mit erfolgreichen Bienen über den Schwänzeltanz. Auf diese Weise wird die Kommunikation innerhalb des Bienenvolks abgebildet und somit die Information über ergiebige Futterquellen geteilt.

Die Nahrungsquellen können von den Bienen angefliegen werden und stellen das zu sammelnde Futter bereit. Jeder Futterquelle besitzt eine Güte anhand derer die Bienen eine Klassifizierung der unterschiedlichen Futterquellen vornehmen können. Ferner besitzt eine Nahrungsquelle eine Anzahl an verfügbarer Nahrung. Bienen können Nahrung aus einer Nahrungsquelle ernten, bis die Nahrungsquelle vollständig erschöpft ist. Abgeerntete Nahrungsquellen werden von Bienen nicht mehr angefliegen, wodurch sich eine sich dynamisch ändernde Priorisierung von Nahrungsquellen ergibt.

Table 3-2: Statusvariablen der Entität Biene

Name	Typ	Beschreibung
x	Float	X - Position im Suchraum
y	Float	Y - Position im Suchraum
hive	Objekt	Bienenstock welcher der Biene zugeordnet ist
occupation	Enum	Aufgabe die Biene aktuell ausführt (Scout, Employed, Onlooker, Dancer)
action	Enum	Aktion die eine Biene innerhalb ihrer Aufgabe aktuell ausführt
speed	Float	Fluggeschwindigkeit der Biene
orientation	Float	Richtungsorientierung der Biene
capacity	Integer	Nahrungsmenge die Biene aktuell trägt
foodsource	Objekt	Futterquelle an der Biene Nahrung sammelt
watchfloor	Objekt	Tanzfläche bei der zugeschaut wird
seen_dances	Dictionary	Sammlung von Tanzinformationen die ein Onlooker gesammelt hat
steps	Integer	Anzahl der Schritte (Ticks) bevor Biene zum Bienenstock zurückkehren muss
dance_counter	Integer	Counter wie lange die Biene tanzen darf
foodsource_pos	Tupel	Position der Futterquelle die gefunden wurde
foodsource_units	Integer	Restliche Units die eine Futterquelle zum Zeitpunkt der Ernte hatte
foodsource_sugar	Integer	Zuckerwert der Futterquelle
dancefloor	Objekt	Tanzfläche auf der die Biene tanzt
success	Integer	Counter wie oft Biene hintereinander erfolgreich Futter gesammelt hat
dance_probability	Integer	Wahrscheinlichkeit, dass Biene tanzt
start_point	Vector2	Startpunkt an dem die Waggle Phase des Tanzes beginnt
end_point	Vector2	Endpunkt an dem die Waggle Phase des Tanzes endet
return_angle	Integer	Bestimmt den Zielrotationswinkel zur Tanzfläche während der Return Phase des Tanzes
dance_direction	Boolean	Entscheidet, ob Return Phase des Tanzes im oder gegen den Uhrzeigersinn ausgeführt wird
size	Integer	Durchmesser der visuellen Bienen-Darstellung
image	Klasse	Pygame.Surface für die visuelle Darstellung des Sprites
radius	Float	Radius der visuellen Bienen-Darstellung
rect	Klasse	Pygame.Rect für die Berechnung von Kollisionen

Table 3-3: Statusvariablen der Entität Bienenstock

Name	Typ	Beschreibung
x		X - Position im Suchraum
y		Y - Position im Suchraum
food_count	Integer	Anzahl der Nahrung im Bienenstock
size	Integer	Durchmesser der visuellen Bienenstock-Darstellung
image	Klasse	Pygame.Surface für die visuelle Darstellung des Sprites
radius	Float	Radius der visuellen Bienenstock-Darstellung
rect	Klasse	Pygame.Rect für die Berechnung von Kollisionen
bees	Klasse	Gruppe die alle Objekte vom Typ Biene enthält
scout_bees	Klasse	Gruppe die alle Objekte vom Typ Sucherinnen enthält
employed_bees	Klasse	Gruppe die alle Objekte vom Typ Sammlerinnen enthält
onlooker_bees	Klasse	Gruppe die alle Objekte vom Typ Zuschauer enthält
dancer_bees	Klasse	Gruppe die alle Objekte vom Typ Tänzerinnen enthält
dancefloor_list	List	Liste der Tanzflächen die sich im Hive befinden
algorithm	Klasse	Bestimmt den Prozess der Ermittlung der Tanzwahrscheinlichkeit

Table 3-4: Statusvariablen der Entität Tanzfläche

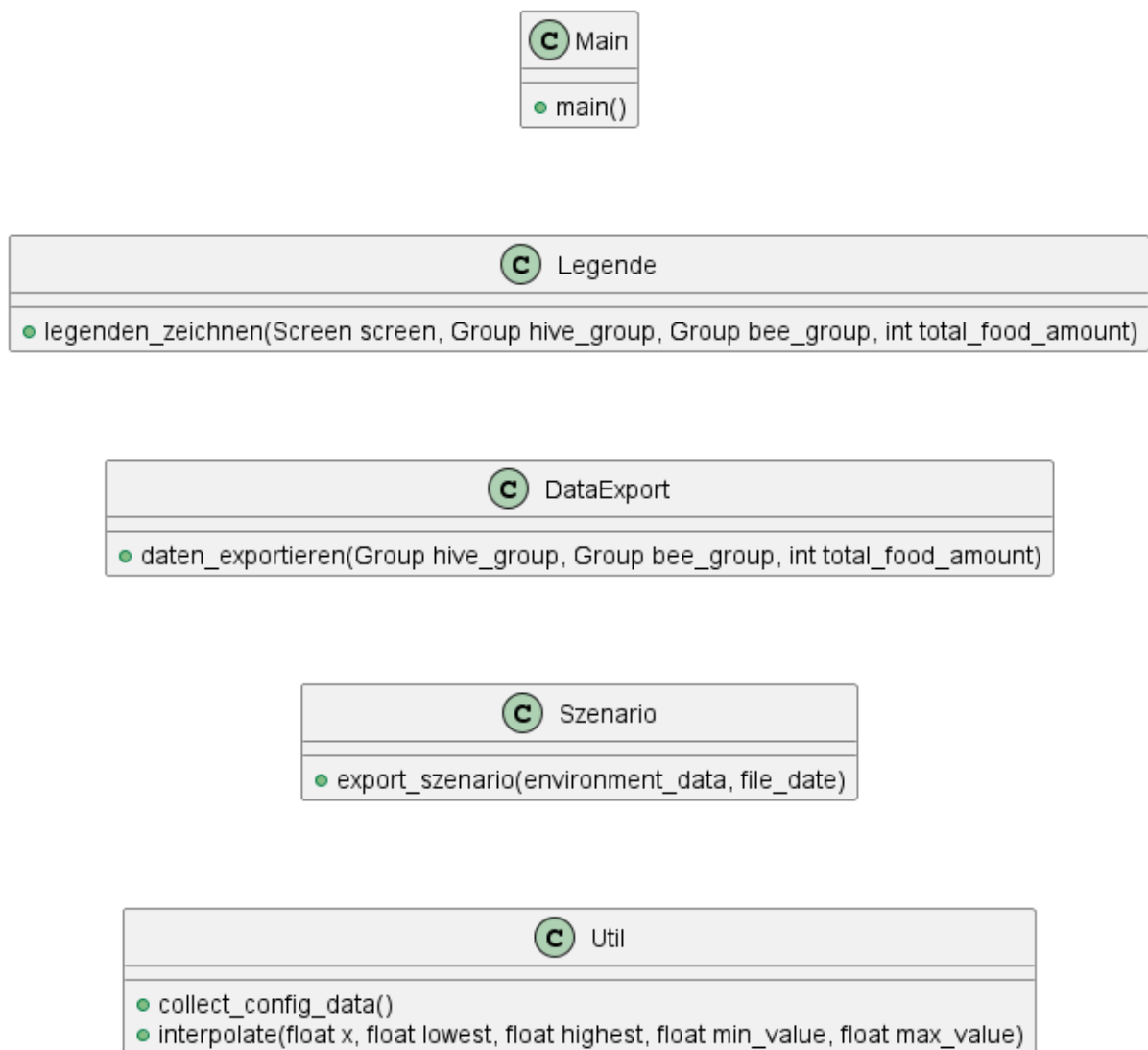
Name	Typ	Beschreibung
X		X - Position im Suchraum
Y		Y - Position im Suchraum
image	Klasse	Pygame.Surface für die visuelle Darstellung des Sprites
radius	Float	Durchmesser der visuellen Bienenstock-Darstellung
rect		Pygame.Rect für die Berechnung von Kollisionen
dancer	Klasse	Biene die auf der Tanzfläche tanzt
onlookers	List	Liste von Bienen die dem Tanz an der Tanzfläche zuschauen

Table 3-5: Statusvariablen der Entität Nahrungsquelle:

Name	Typ	Beschreibung
X		X - Postition im Suchraum
Y		Y - Postition im Suchraum
units	Tupel	Anzahl der Nahrung an der Futterquelle
sugar	Tupel	Zuckergehalt der Futterquelle
image	Pygame.Surface	Pygame.Surface für die visuelle Darstellung des Sprites
radius	Float	Durchmesser der visuellen Bienenstock-Darstellung
rect	Pygame.Rect	Pygame.Rect für die Berechnung von Kollisionen
label_font	Pygame.Font	Font für die Beschriftung der Futterquellen
distance_to_hive	Tupel	Entfernung zum Bienenstock

3.5. Prozessübersicht

Im Laufe der Simulation können die Bienen verschiedene Aufgaben innerhalb des Prozesses der Futtersuche übernehmen. Es gibt keine zentrale Intelligenz im Bienenstock, die den Bienen Aufgaben vergibt. Die Entscheidung, welche Aufgabe ein Individuum gerade übernimmt, ist dynamisch und wird durch die Umgebungsbedingungen, von der Biene selbst und die Interaktion mit anderen Bienen gesteuert. Abbildung 3.1 zeigt eine Übersicht über die gekoppelten Prozesse innerhalb der simulierten Nahrungssuche.



E Occupation
SCOUT
EMPLOYED
ONLOOKER
RETURNING
DANCER

E Action
SCOUTING
WANDERING
LOOKING
FORAGING
RETURNING
DANCE_WAGGLE
DANCE_RETURN
WAITING

E Algorithm
SUGAR
REPETITION
NONE

C Color
○ <u>Tuple WHITE</u>
○ <u>Tuple GREY</u>
○ <u>Tuple YELLOW</u>
○ <u>Tuple GREEN</u>
○ <u>Tuple DARK GREEN</u>
○ <u>Tuple BLACK</u>
○ <u>Tuple COLOR BEE SCOUT</u>
○ <u>Tuple COLOR BEE EMPLOYED</u>
○ <u>Tuple COLOR BEE ONLOOKER</u>
○ <u>Tuple COLOR BEE DANCER</u>

C Config
○ <u>int FRAMES PER SECOND</u>
○ <u>int SCREEN HEIGHT</u>
○ <u>int SCREEN WIDTH</u>
○ <u>int SIMULATION WIDTH</u>
○ <u>int BEES SCOUT</u>
○ <u>int BEES ONLOOKER</u>
○ <u>int MAX BEES SCOUT</u>
○ <u>int MAX BEES DANCER</u>
○ <u>int FOOD COUNT</u>
○ <u>int MAX SUGAR</u>
○ <u>int MIN SUGAR</u>
○ <u>int MAX UNITS</u>
○ <u>int MIN UNITS</u>
○ <u>int MAX TIME</u>
○ <u>int MIN RANGE FOOD TO HIVE</u>
○ <u>int MAX VELOCITY BEE</u>
○ <u>int MIN VELOCITY BEE</u>
○ <u>int WALKING SPEED</u>
○ <u>int BEE VISION</u>
○ <u>int BEE MAX CAPACITY</u>
○ <u>int REDUCE SPEED WHEN CARRY</u>
○ <u>int MAX STEP COUNTER BEES</u>
○ <u>int DANCETIME PER UNIT</u>
○ <u>int DANCEFLOOR RADIUS</u>
○ <u>int DANCEFLOOR CAPACITY</u>
○ <u>int MIN DANCE PROBABILITY</u>
○ <u>int MAX DANCES WATCHED</u>
○ <u>int MIN DANCES WATCHED</u>
○ <u>int EXPORT ENVIRONMENT</u>
○ <u>int IMPORT ENVIRONMENT FILE</u>
○ <u>int EXPORT SIMULATION</u>
○ <u>int EXPORT COMPLETE BEE GROUP</u>
○ <u>int EXPORT DATA INTERVALL</u>

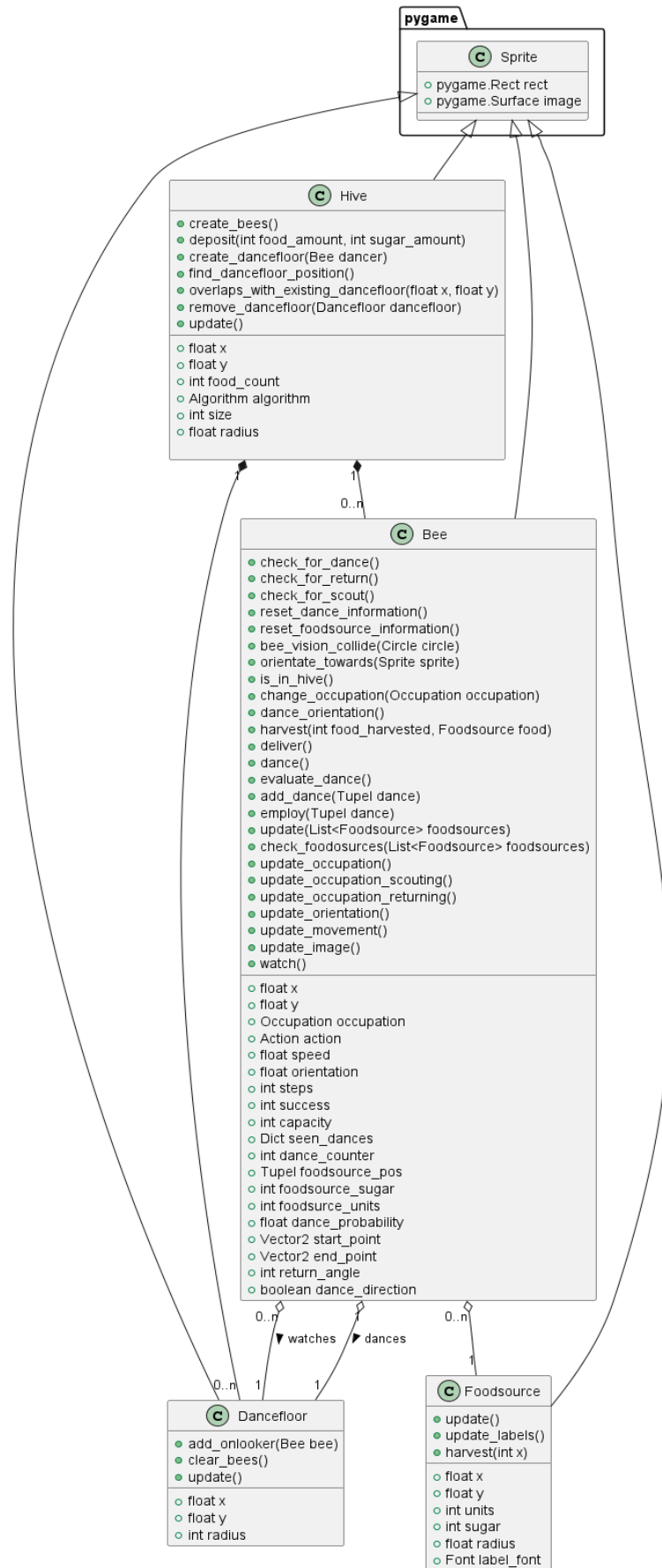


Abbildung 3.1: Prozessübersicht der simulierten Nahrungssuche

3.6. Grundprinzipien

Im folgenden Abschnitt werden einige biologische Verhaltensweisen der Bienen und deren Implementierung in das Modell genauer beschrieben.

3.6.1. Tanzwahrscheinlichkeit

Das Modell enthält zwei verschiedene Konzepte zur Bestimmung der Tanzwahrscheinlichkeit. Beide Konzepte basieren auf in der Natur beobachteten Verhaltensweisen und können anhand von wissenschaftlicher Literatur belegt werden.

a) Tanzwahrscheinlichkeit in Abhängigkeit der Futterqualität

Die Tanzwahrscheinlichkeit ermittelt sich in der Natur anhand der Qualität bzw. der Attraktivität der gefundenen Nahrungsquelle (Tautz, 2022). Wenn eine Sucherin oder eine Sammlerin erfolgreich eine Futterquelle besucht hat und dort Nektar ernten konnte, dann wird sie diesen zurück zum Bienenstock bringen und dann entscheiden, ob Sie durch einen Schwänzeltanz für die eben besuchte Futterquelle wirbt oder nicht.

Dies wird in dem Modell durch eine Interpolation der Tanzwahrscheinlichkeit anhand der Parameter MAX_SUGAR, MIN_SUGAR und dem tatsächlichen Zuckerwert der besuchten Futterquelle nachgebildet. Wenn die Futterquelle nur Futter von sehr schlechter Qualität enthielt, der Zuckergehalt also den Wert MIN_SUGAR hatte, dann liegt die Wahrscheinlichkeit dafür, dass für diese Futterquelle getanzt wird auch sehr niedrig. Diese minimale Tanzwahrscheinlichkeit wird in der Simulation durch einen eigenen Parameter MIN_DANCE_PROBABILITY angegeben. Wird eine Futterquelle mit dem maximalen Zuckergehalt besucht, dann tanzt die Biene in jedem Fall. Die Wahrscheinlichkeit liegt dann also bei 100%. Für alle Fälle zwischen diesen beiden Extremwerten steigt die Wahrscheinlichkeit linear mit dem Zuckergehalt an.

b) Tanzwahrscheinlichkeit in Abhängigkeit des Sammelerfolgs

Aufgrund von empirischen Untersuchungen wurde festgestellt das Bienen in der Natur meistens nach der sogenannten 40/60/80% Tanz-Strategie vorgehen (Van Nest & Moore, 2012). Diese Strategie besagt, dass die Tanzwahrscheinlichkeit einer Biene zunimmt, solange dieser an der gleichen Nahrungsquelle erfolgreich ist. In ihrer ursprünglichen Form ist die Tanzwahrscheinlichkeit der Biene von ihrem Sammelerfolg über einen gesamten Tag abhängig. Wenn die Biene am ersten Tag ihres Ausflugs erfolgreich Nahrung sammeln konnte, besteht nach dieser Strategie eine 40%ige Wahrscheinlichkeit, dass die Biene ihre Nahrungsquelle über den Schwänzeltanz an andere Bienen weitergibt. Wenn die Biene am darauffolgenden Tag weiterhin an der Nahrungsquelle erfolgreich ist, steigt die Tanzwahrscheinlichkeit auf 60% und bei einem weiteren Sammelerfolg am dritten Tag auf 80%.

Da das Modell keine zeitliche Aufschlüsselung nach expliziten Zeitschritten (z.B. Stunden) vorsieht, wurde in dem Modell eine leicht vereinfachte Version der 40/60/80% Strategie implementiert. In der Modellversion ist die Wahrscheinlichkeit für den Tanz lediglich von hintereinander stattfindenden, erfolgreichen Sammelflügen abhängig, ohne dass eine zeitliche Komponente in Betracht gezogen wird.

Über den Hive Parameter algorithm kann eingestellt werden, anhand welchen Konzepts die Bienen die Tanzwahrscheinlichkeit ermitteln.

3.6.2. Ermittlung der maximalen Sucherinnen

In eine Untersuchung zum Verhältnis von Sucherinnen und Zuschauerinnen machte Seeley 1983 die Beobachtung, dass ein Bienenvolk die Anzahl der Sucherinnen in Abhängigkeit der Tänzerinnen dynamisch anpasst. Je mehr Tänzerinnen in einem Bienenstock aktiv sind, desto weniger Sucherinnen werden rekrutiert. Diese Rückkopplung ist sinnvoll, da eine hohe Anzahl von Tänzerinnen auf eine hohe Anzahl an bekannten Futterquellen hindeutet. Aus einer energetischen Betrachtung macht es in so einer Situation für ein Bienenvolk nicht viel Sinn, Kapazitäten auf der Suche nach neuen Futterquellen zu verbrauchen. Sind hingegen wenig Tänzerinnen aktiv, lässt dies auf eine Limitation der Futterquellen schließen, in diesem Fall ist es demnach sinnvoll die Anzahl der Sucherinnen zu erhöhen, um weiter Futterquellen zu erschließen.

Anhand der Angaben aus Seeley, 1983 wurde eine lineare Interpolation berechnet (s. Abbildung 3-2) anhand derer im Modell die maximale Anzahl der Sucherinnen in Abhängigkeit der Tänzerinnen berechnet und als Grenzwert eingesetzt wird.

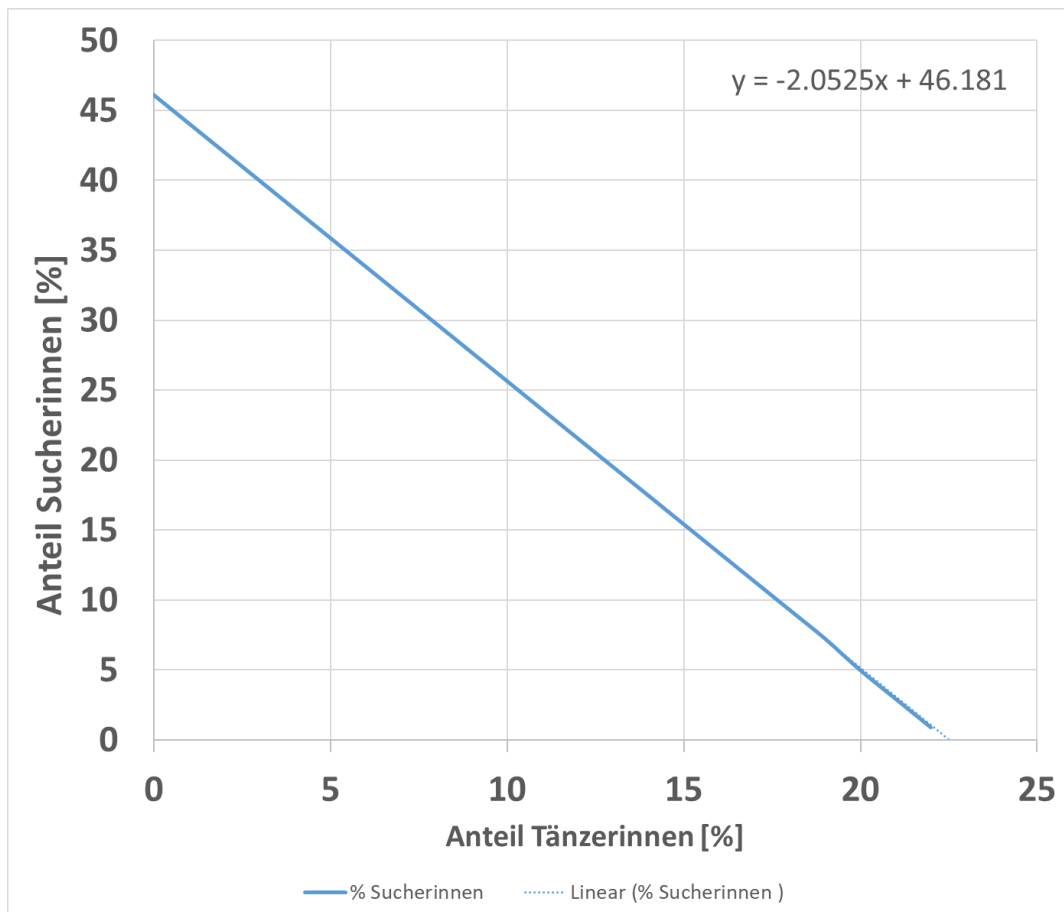


Abbildung 3.2: Verhältnis des prozentualen Anteils der Sucherinnen in Abhängigkeit des prozentualen Anteils der Tänzerinnen

3.6.3. Tanzfläche und Schwänzeltanz

Ist eine der Tanzflächen im Bienenstock frei, entscheidet sich eine Biene dazu, nach der oben beschriebenen Wahrscheinlichkeit den Schwänzeltanz auszuführen. Dadurch wird die Tanzfläche belegt und andere Bienen in nächster Nähe können den Tanz beobachten. Größe und Kapazität einer Tanzfläche wird über die Parameter „DANCEFLOOR_RADIUS“ und „DANCEFLOOR_CAPACITY“ begrenzt. Diese sind entscheidend dafür, ob eine andere Biene den Tanz beobachten kann.

3.6.4. Berechnung der Tanzzeit

Ist eine der Tanzflächen im Bienenstock Die Dauer eines Schwänzeltanzes berechnet sich aus der Entfernung zwischen dem Bienenstock und der Futterquelle. Diese Berechnung ist an das natürliche Verhalten der Bienen angelehnt bei dem die Tanzzeit mit der Entfernung zunimmt (Taus, 2022). Die Distanz berechnet sich in unserer Simulation in Pixeln. Die Tanzdauer bestimmt sich dann anhand dieser Distanz und dem Parameter DANCETIME_PER_UNIT. Wichtig ist, dass die Tanzzeit nur gezählt wird, wenn die Biene sich in der Waggle-Phase des Tanzes befindet. Kehrt sich

während der Return-Phase zu ihrer Ausgangsposition zurück, tickt der `dance_counter` währenddessen nicht herunter.

3.6.6 Übergabe der Tanzinformationen

Trifft eine Zuschauerin, während Sie durch den Bienenstock wandert auf eine Tanzfläche an der noch Plätze frei sind, wird sie anhalten und sich den dort stattfindenden Tanz anschauen. Dabei werden die Informationen der Futterquelle während des Tanzes von der tanzenden Biene an die zuschauende Biene übergeben. Die tanzende Biene kennt die Position der Futterquelle (`foodsource_pos`), den Restbestand der Futterquelle zu dem Zeitpunkt, an dem Sie diese zuletzt besucht hat (`foodsource_units`), sowie den Zuckergehalt der Futterquelle (`foodsource_sugar`). Zusätzlich hält sie eine Referenz auf die Futterquelle an sich (`foodsource`) für die Kollisionsberechnung. Am Ende des Tanzes, wird der Inhalt der Variable an die zuschauende Biene übergeben. Die Zuschauerin speichert die Informationen in dem Dictionary `seen_dances`.

3.6.5. Rekrutierung der Zuschauerinnen

In der Natur ist es so, dass sich Zuschauerinnen im Bienenstock zwischen 5 und 8 Tänze anschauen, bevor Sie sich für eine Futterquelle entscheiden, zu der sie ausfliegen möchten (Tautz, 2022). Diese Grenzwerte werden auch in der Simulation berücksichtigt. Sobald eine Zuschauerin 5 Tänze gesehen hat, besteht eine Chance, dass Sie sich für eine der gesehen Futterquellen anwerben lässt und den Bienenstock als Sammlerin für eben diese Futterquelle verlässt. Spätestens nach dem 8 gesehen Tanz wird die Zuschauerin aber definitiv den Bienenstock verlassen, um Nektar zu sammeln. Um den Umstand abzubilden, dass die Simulation die Größenverhältnisse eines echten Bienenstockes nicht 1:1 abbildet sind die Grenzwerte für die zu beobachtenden Tänze durch Konfigurationsparameter anpassbar.

Sobald die minimale Anzahl zu beobachtende Tänze erreicht wurde, beträgt die Wahrscheinlichkeit, dass die Zuschauerin sich für eine Futterquelle entscheidet, 50%. Nach der maximalen Anzahl 100%. Zwischen diesen Grenzwerten steigt die Wahrscheinlichkeit zum Ausflug mit jedem gesehenen Tanz linear an.

3.6.7 Entdecken und Ernten von Nahrungsquellen

Sobald eine Biene in der Nähe einer Nahrungsquelle kommt, wird diese geerntet, insofern die Biene noch freie Tragkapazitäten für Nahrung hat. Die aktuelle Kapazität einer Biene wird über die Statusvariable „`capacity`“ getrackt und über den Parameter „`BEE_MAX_CAPACITY`“ begrenzt. Ist die maximale Kapazität der Biene erreicht, fliegt sie zurück zum zugehörigen Bienenstock.

Der Sichtradius der Biene wird über den Parameter „`BEE_VISION`“ realisiert. Dieser ist sehr klein, da Bienen generell eine kurze Sicht haben.

Beim Ernten werden die Informationen der Nahrungsquelle, die bereits im Kapitel 3.5.5 beschrieben wurden, an die sammelnde Biene übergeben.

3.7 Initialisierung

3.7.1 Initialisierung der Umgebung

Zu Beginn der Simulation werden Bienenstöcke und Nahrungsquellen zufällig oder anhand eines eingelesenen Szenarios in der Landschaft erstellt. Ein Überlappen von Nahrungsquellen ist explizit nicht im Code verhindert worden, da sich Nahrungsquellen von Bienen in der Natur in Form von Blumenfeldern darstellen und in Anzahl, Größe und Dichte stark variieren. Es wird lediglich verhindert, dass Nahrungsquellen nicht zu nah an einem Bienenstock erstellt werden. Die Anzahl an zu generierenden Nahrungsquellen wird über den Parameter „FOOD_COUNT“ eingestellt und die initial enthaltene Nahrungsmenge, sowie der Zuckergehalt innerhalb der Parameter „MIN_UNITS“ und „MAX_UNITS“, sowie „MIN_SUGAR“ und „MAX_SUGAR“ festgesetzt.

3.7.2 Initialisierung der Community

Die Anzahl der initialisierten Bienen wird über die beiden Parameter BEES_SCOUT und BEES_ONLOOKER bestimmt.

Mit Hilfe dieser beiden Parameter werden für jeden Bienenstock die zugehörigen Bienen erstellt. Jede Biene startet in ihrem Bienenstock und bekommt initial die Koordinaten des Bienenstocks übergeben, sodass sie zu diesem zugehörig ist und zu jeder Zeit wieder zurückfliegen kann.

Späherinnen fliegen nach der Initialisierung mit zufälligen Richtungskoordinaten los, um neue Nahrungsquellen zu finden, während Sammlerinnen im Bienenstock auf tanzende Bienen warten.

4 Simulation

Im folgenden Kapitel sollen verschiedene Simulationen mit dem entworfenen Multiagentensystem durchgeführt werden. Dabei sollen bestimmte Eingabeparameter der Simulation gezielt modifiziert werden. Dies führt zu unterschiedlichen Simulationsergebnissen, wobei schon kleinste Anpassungen große Auswirkungen auf die Effizienz der Nahrungssuche haben können. Diese Parameteranpassungen wurden ausgewählt um die Auswirkung auf den Algorithmus genauer zu untersuchen, und so die Nahrungssuche zu optimieren.

Um die unterschiedlichen Simulationsergebnisse miteinander vergleichen zu können, wurden verschiedene Funktionen in den Code implementiert:

Umgebungs-Exportfunktion: Diese Funktion ermöglicht es, eine zufällig initialisierte Umgebung (beschrieben in 3.7) in einer Komma separierten Datei abzuspeichern. Entsprechend werden in dieser Datei die Informationen zu allen existierenden Futterquellen und Bienenstöcken gesichert. Die entsprechende Datei wird mit einem Zeitstempel (Datum und Uhrzeit) benannt und kann zu einem späteren Zeitpunkt erneut für eine Simulation eingelesen werden. Auch ein manuelles Bearbeiten der Umgebung wird auf diese Weise ermöglicht, um zum Beispiel Positionen oder Futterwerte von Futterquellen modifiziert zu können. Somit kann durch diese Funktion die gleiche Umgebung für mehrere Simulationen verwendet werden.

Aufzeichnungsfunktion: Um während einer Simulation informative Parameter zu erfassen, kann die Aufzeichnungsfunktion verwendet werden. Dabei handelt es sich um ein Event welches periodisch die Parameterwerte der Simulation abtastet und somit einen kontinuierlichen Datensatz (Dataframe) erstellt. Das Intervall der Abtastung kann dabei zu Beginn nach Belieben gewählt werden (Standartwert = 1000ms). Die Funktion ermöglicht zwei verschiedene Umfänge der Aufzeichnung:

- a) reduzierte Parameter-Aufzeichnung:
Dabei erfasst die Funktion zu jedem Intervall den entsprechenden Zeitstempel, die bereits gesammelte Futtermenge, sowie die Summen der Bienen welche eine entsprechende Aufgabe verfolgen (Kategorisierung nach Occupation).
- b) Vollständige Parameter-Aufzeichnung:
Zusätzlich zu Zeitstempel und gesammelter Futtermenge, erstellt dieser Modus eine vollständige Aufzeichnung des gesamten Bienenvolks. Dabei werden zu jeder einzelnen Biene die Parameter Aufgabe, freie Kapazität, Zielfutterquelle, sowie ein Zähler für die jeweiligen Sammel-Erfolge der Biene erfasst. Eine vollständige Parameter-Aufzeichnung ermöglicht später eine umfassende Reproduktion der entsprechenden Simulation

Simulations-Exportfunktion: Die in der Aufzeichnungsfunktion aufgezeichneten Daten können mit dieser Funktion am Ende der Simulation in einem Microsoft-Excel-

Format exportiert werden. Dadurch wird eine nachträgliche Analyse der Simulation ermöglicht, mit allen Funktionen welche Microsoft-Excel bietet (Datenfilter, Diagramme, Pivot-Table, ...). Zusätzlich zu den eigentlichen Simulationsparametern wird auch die zugehörige Konfigurationsdatei in einem separaten Excel-Tab exportiert, was im späteren Verlauf präzisere Aussagen über die Konfiguration der Simulation ermöglichen kann.

Die in diesem Kapitel beschriebenen Funktionen dienen lediglich zur Unterstützung der Simulationen und sind dadurch kein essentieller Bestandteil des Multiagentensystems. Die entsprechenden Einstellungen der Funktionen können durch die anwendende Person nach Belieben in der Konfigurations-Datei des Systems eingestellt werden.

4.1 Parameteranpassungen und Annahmen

Bei der Simulation eines Bienenvolks als MAS ergeben sich zahlreiche Möglichkeiten entsprechende Testszenarien aufzusetzen und miteinander zu vergleichen. Im Rahmen der Entwicklung des MAS wurden unterschiedliche Testszenarien überlegt und diskutiert. In einem ersten Entwurf, war zum Beispiel angedacht Hindernisse (Mauern, Häuser oder Autos) in der Umgebung zu platzieren und hier das Verhalten entsprechende Verhalten der Bienen zu analysieren. Da sich hier aber kein wirklicher Mehrwert für die Simulation selbst bieten würde, (da die Bienen die meisten Hindernisse einfach überfliegen könnten) wurde auf diese Testszenarien verzichtet. Die nachfolgenden Tests bieten damit nur eine Art möglicher Untersuchungen.

4.2 Beobachtungen

4.2.1. Priorisierung

Wie bereits in Kapitel 3.6.1 erläutert, wurden im MAS zwei verschiedene Ansätze zur Tanzwahrscheinlichkeit implementiert (Qualität oder Sammelerfolg an der Futterquelle). Um die Umsetzung dieser Ansätze zu überprüfen, wurden zwei entsprechender Test aufgesetzt. In der Umgebung des Bienenstocks befinden sich, in diesem Testcase lediglich zwei Futterquellen. Diese Futterquellen haben zwar den gleichen Abstand zum Bienenstock und einen Identischen Futterwert, jedoch unterscheiden sich die beiden Futterquellen in ihrem Zuckergehalt. So hat die Futterquelle 1 lediglich einen Zuckergehalt von 1 SV (Sugar Value) während Futterquelle 2 einen Zuckergehalt von 2 SV hat. Im Folgenden wurde jeweils ein Test mit den implementierten Tanzwahrscheinlichkeiten durchgeführt. Die Tanzwahrscheinlichkeit in Abhängigkeit der Futterqualität wird dabei als Testcase „Sugar“ bezeichnet. Die Tanzwahrscheinlichkeit in Abhängigkeit des Sammelerfolgs wird als Testcase „Repetition“ bezeichnet.

Ergebnisse Testcase Sugar:

In Abbildung 4.1 ist zu beobachten, dass die Bienen zu Beginn der Simulation sehr gleichmäßig auf die beiden Futterquellen verteilt sind. Ungefähr ab Sekunde 10, wird Futterquelle 2 (7 SV) jedoch eindeutig priorisiert. Dies kann darauf zurückgeführt werden, dass ab diesem Zeitpunkt die Tänze im Bienenstock vermehrt über die Futterquelle 2 handeln, da diese einen höheren Zuckerwert (Qualität) hat.

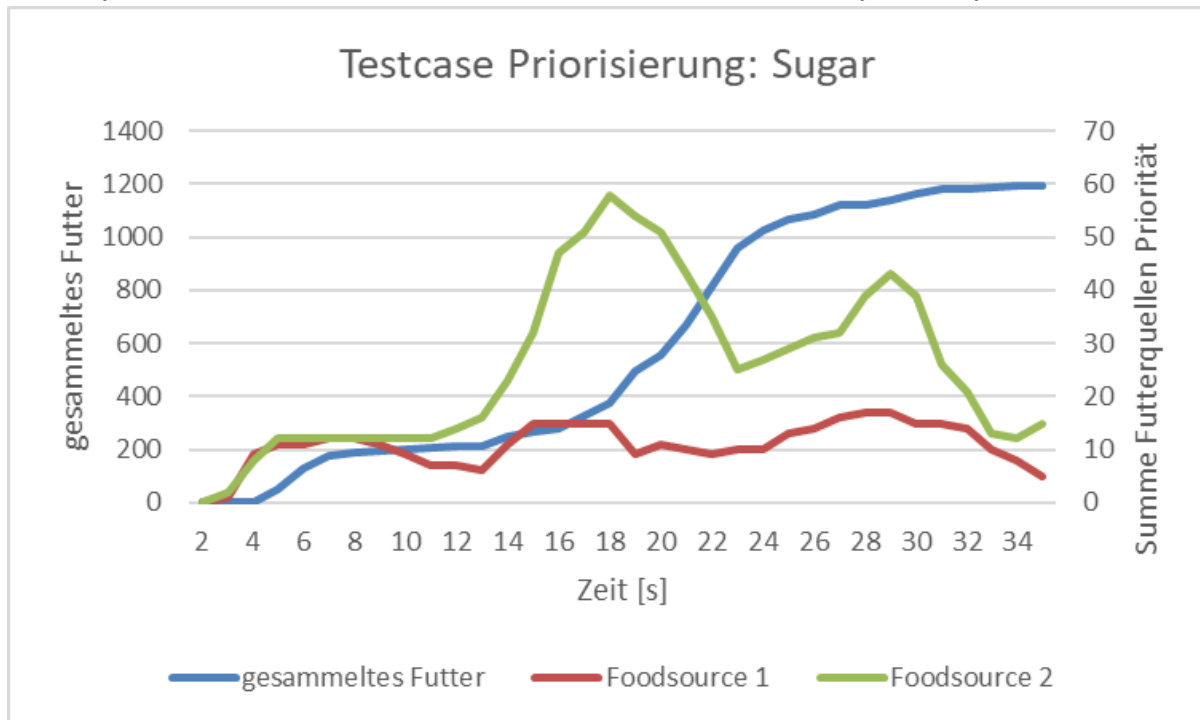


Abbildung 4.1: Testcase Priorisierung mit Tanzalgorithmus Sugar

Ergebnisse Testcase Repetition:

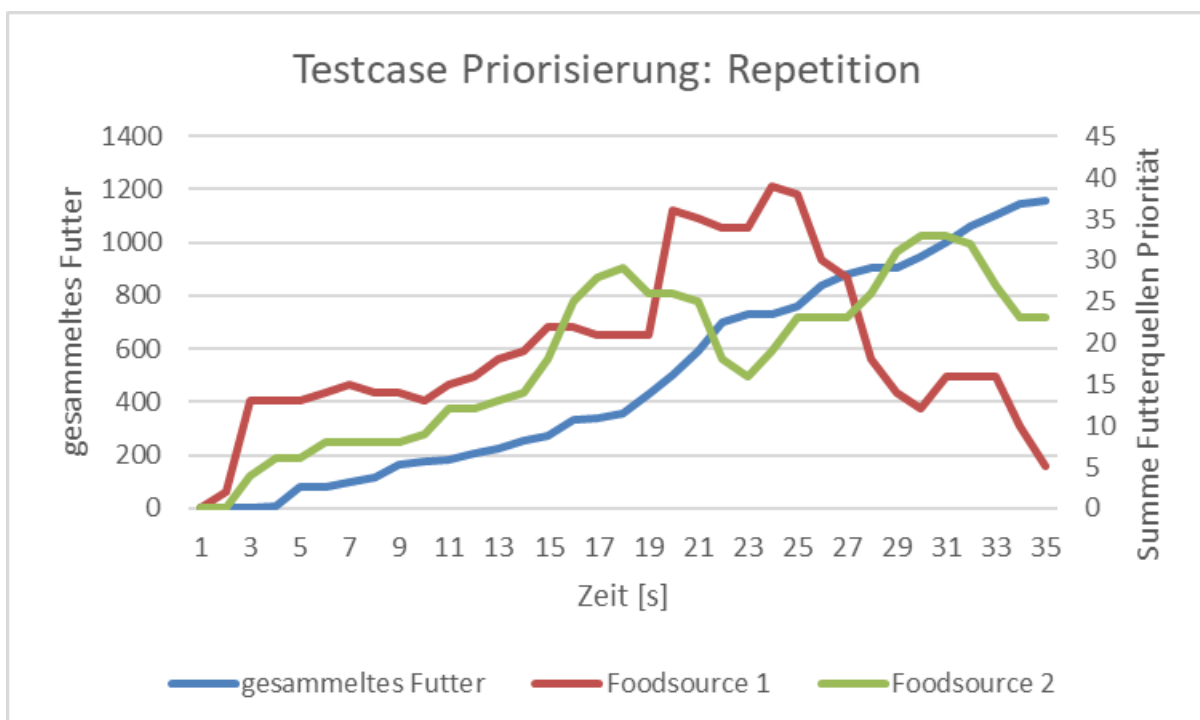


Abbildung 4.2: Testcase Priorisierung mit Tanzalgorithmus Repetition

Im Vergleich zum zuvor verwendeten Sugar-Algorithmus ist hier klar zu erkennen, dass Futterquelle 1 nahezu dauerhaft die bevorzugte Futterquelle ist, auch wenn Futterquelle 2 mit einer deutlich höheren Qualität (7SV) verfügbar wäre. Dies ist im Falle des Repetition Algorithmus darauf zurück zu führen, dass Futterquelle 1 zuerst durch Scout Bienen entdeckt und auf der Tanzfläche bevorzugt kommuniziert wurde.

4.2.2. Algorithmen

Um die verwendeten Algorithmen weiter zu untersuchen wurde ein weiteres Testszenario aufgebaut. Hierfür wurden die beiden Algorithmen in einer identischen Umgebung und unter den gleichen Bedingungen angewendet. Zusätzlich wurde eine weitere Simulation mit dem Algorithmus „NONE“ durchgeführt. Bei dieser werden alle Bienen als Scouts initialisiert und der Schwänzeltanz somit komplett ausgesetzt. Die Umgebung in der die Bienen ausgesetzt werden enthält eine hohe Dichte an Futterquellen.

Ergebnisse Testcase Algorithmen:

Interessanterweise ist zu beobachten, dass die Simulation ohne konkreten Schwänzeltanz-Algorithmus den besten Ernteerfolg hat. So ist diese Simulation ungefähr doppelt so schnell wie die Simulationen welche mit den Schwänzeltanz-Algorithmen Sugar oder Repetition durchgeführt wurden. Dies unterstützt die These von aktuellen Forschungsberichten, wonach der Schwänzeltanz unter menschlich beeinflussten Umgebungen keinen direkten Effizienzgewinn garantieren kann. Gründe hierfür können unter anderem eine erhöhte Futterdichte im Bereich des Bienenvolks sein, wie zum Beispiel landwirtschaftliche Anbaugebiete (l'Anson Price et al., 2019).

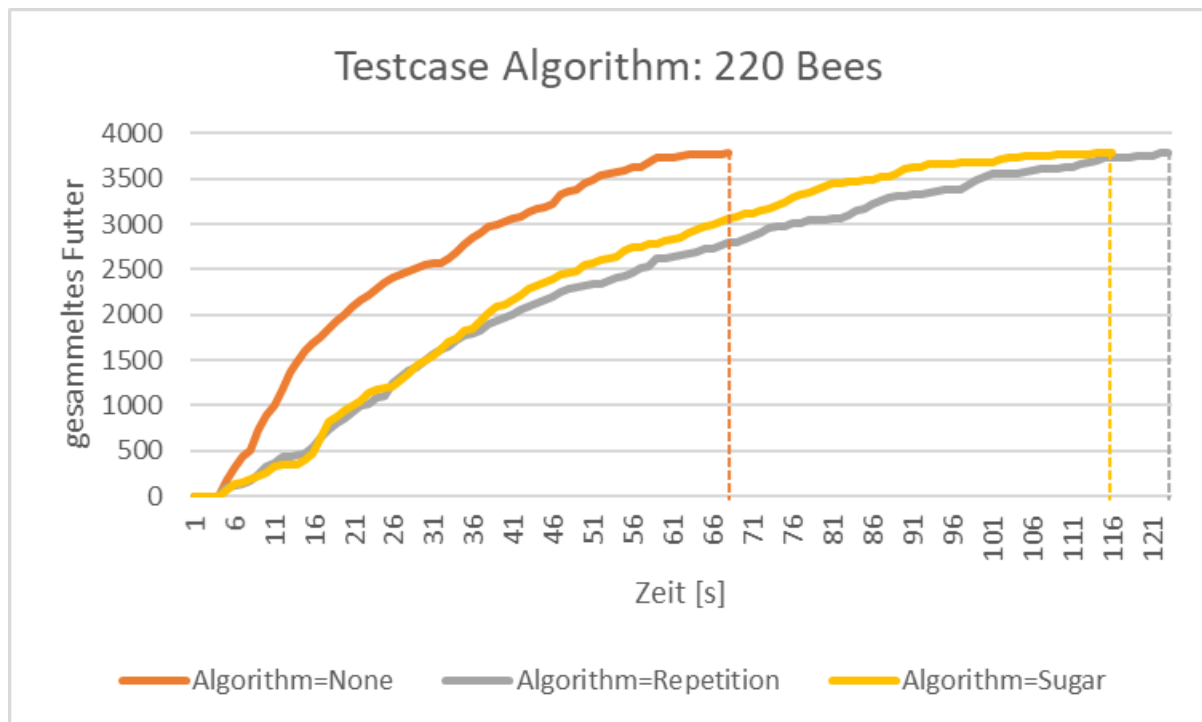


Abbildung 4.3: Testcase Algorithm

4.2.3. Kapazität

In einem weiteren Testszenario wurde die Futterkapazität der Bienen variiert. Jede einzelne Biene ist in den Tests in der Lage entweder ein, zwei drei oder vier Futtereinheiten (FE) tragen zu können. Es sollte untersucht werden, wie sich eine veränderte Futterkapazität auf die Ernte der Futterquellen auswirkt. Analog zum vorangehenden Test wird auch hier die Umgebung und sämtliche übrigen Parameter für jede Simulation konstant gehalten.

Ergebnisse Testcase Kapazität:

Es kann beobachtet werden, dass die Simulation mit einer Kapazität von 4 FE um zwei Drittel schneller ist als die Simulation mit einer Kapazität von 1 FE (Futtereinheiten). Jedoch reicht bereits eine Verdopplung der Futterkapazität von 1 FE auf 2 FE aus um die Ernte um ein Drittel zu beschleunigen. Dieser Zusammenhang ist in

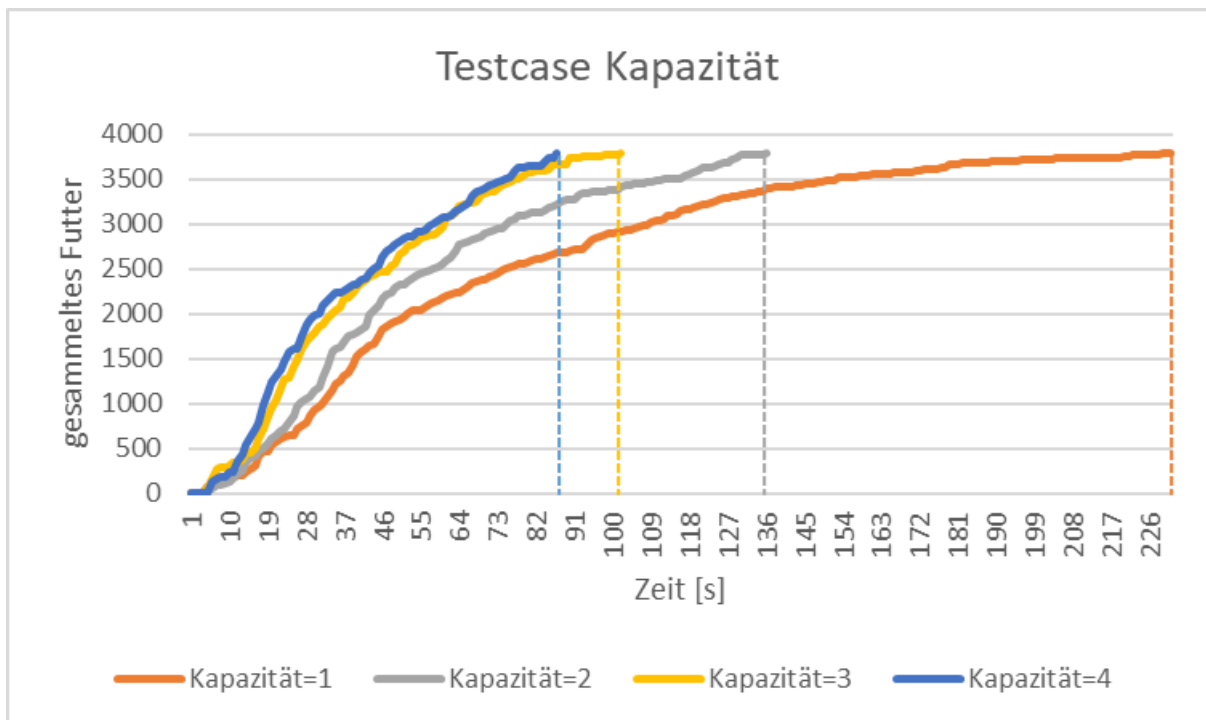


Abbildung 4.4 sehr gut zu erkennen.

Die Erhöhung der Tragkapazität von Nahrung pro Biene erhöht die Effizienz des Algorithmus maßgeblich, die Bienen dadurch weniger Strecke zurücklegen müssen. Die Kapazität kann jedoch nicht beliebig erhöht werden.

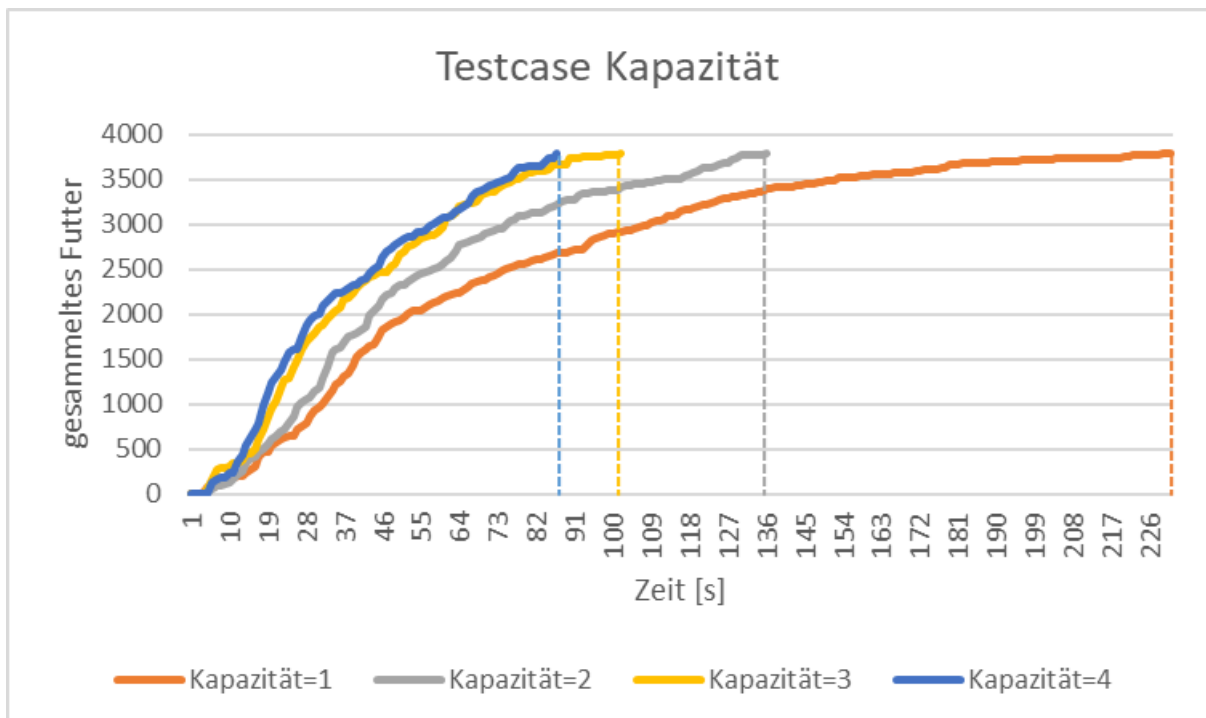


Abbildung 4.4: Testcase Kapazität

4.3. Zusammenfassung

Nach der Durchführung der Simulationen ist zu erkennen, dass wissenschaftliche Grundlagen über die Futtersuche von Bienen in einem Multiagentensystem abgebildet

werden können (siehe Simulation 4.2.1 oder 4.2.3). Es ist jedoch zu beobachten, dass Ergebnisse je nach Konfiguration der Eingabe-Bedingungen variieren können. So lässt sich zum Beispiel darauf schließen, dass die Ergebnisse von Simulation 4.2.2 durch eine Vergrößerung der Umgebung unterschiedlich ausgefallen wären. Durch eine Vergrößerung der Umgebung würde die Futterdichte sinken. Dies könnte zum Beispiel dazu führen, dass eine Durchführung des Schwänzeltanzes für eine Erhöhte Effizienz bei der Futtersuche sorgen könnte.

5 Fazit

Im Rahmen dieses Fachpraktikums wurde ein Multiagentensystem entworfen und simuliert welches die Futtersuche eines Bienenvolks abbildet. Dabei wurde das System so ausgelegt, dass Simulationen mit sich verändernden Parametern durchgeführt werden können. Um ein möglichst naturgetreues Suchverhalten der Bienen darstellen zu können, wurden verschiedene Such-Algorithmen mittels wissenschaftlicher Literatur behandelt, analysiert und verglichen. Anschließend wurden die entsprechenden Algorithmen in das Multiagentensystem implementiert.

Mithilfe des entwickelten Multiagentensystems ist es möglich unterschiedliche Simulationen zur Bienen-Futtersuche durchzuführen. Dabei werden, die im Umfeld gefundenen Futterquellen (Lösungen) anhand von verschiedenen Qualitätsmerkmalen durch die einzelnen Agenten klassifiziert und vom Multiagentensystem priorisiert. So können zum Beispiel die implementierten Algorithmen auf einer identischen Simulationsumgebung eingesetzt und die entsprechenden Unterschiede verglichen werden. Eine Auswahl dieser Simulationen wurde in dieser Arbeit dargestellt und erläutert. Grundsätzlich zeigen die Ergebnisse, dass das Bienenvolk als dezentralisierte Einheit gesehen werden kann und somit das Verhalten eines Multiagentensystems widerspiegelt. Die Intelligenz des Multiagentensystems kann dabei auf die Kommunikation (Schwänzeltanz) der einzelnen Agenten (Bienen) zurückgeführt werden.

6. Literatur

- Abou-Shaara, H. F. (2014). The foraging behaviour of honey bees, *Apis mellifera*: A review. *Veterinárni Medicina*, 59(1), 1–10. <https://doi.org/10.17221/7240-VETMED>
- Baird, E., Tichit, P., & Guiraud, M. (2020). The neuroecology of bee flight behaviours. *Current Opinion in Insect Science*, 42, 8–13. <https://doi.org/10.1016/j.cois.2020.07.005>
- Corkill, D. D., & Lesser, V. R. (o. J.). *Use of Meta-Level Control for Coordination in a Distributed Problem-Solving Network*.
- Durfee, E. H., & Lesser, V. R. (1989). Chapter 10—Negotiating Task Decomposition and Allocation Using Partial Global Planning. In L. Gasser & M. N. Huhns (Hrsg.), *Distributed Artificial Intelligence* (S. 229–243). Morgan Kaufmann. <https://doi.org/10.1016/B978-1-55860-092-8.50014-9>
- Grimm, V., Berger, U., DeAngelis, D. L., Polhill, J. G., Giske, J., & Railsback, S. F. (2010). The ODD protocol: A review and first update. *Ecological Modelling*, 221(23), 2760–2768. <https://doi.org/10.1016/j.ecolmodel.2010.08.019>
- Hollstein, R. (2023). *Optimierungsmethoden: Einführung in die klassischen, naturanalogen und neuronalen Optimierungen*. Springer Vieweg. <https://doi.org/10.1007/978-3-658-39855-2>
- l’Anson Price, R., Dulex, N., Vial, N., Vincent, C., & Grüter, C. (2019). Honeybees forage more successfully without the “dance language” in challenging environments. *Science Advances*, 5(2), eaat0450. <https://doi.org/10.1126/sciadv.aat0450>
- Karaboga, D. (2005). *An idea based on honey bee swarm for numerical optimization*.

- Seeley, T. D. (1983). Division of labor between scouts and recruits in honeybee foraging. *Behavioral Ecology and Sociobiology*, 12(3), 253–259.
<https://doi.org/10.1007/BF00290778>
- Sushil, S. N., Stanley, J., Hedau, N. K., & Bhatt, J. C. (2013). Enhancing Seed Production of Three Brassica vegetables by Honey Bee Pollination in North-western Himalayas of India. *Universal Journal of Agricultural Research*, 1(3), 49–53. <https://doi.org/10.13189/ujar.2013.010301>
- Sycara, K. P. (1998). Multiagent Systems. *AI Magazine*, 19(2), Article 2.
<https://doi.org/10.1609/aimag.v19i2.1370>
- Sycara, K., Pannu, A., Williamson, M., Zeng, D., & Decker, K. (1996). Distributed intelligent agents. *IEEE Expert*, 11(6), 36–46. IEEE Expert.
<https://doi.org/10.1109/64.546581>
- V. Frisch, K. (1964). *Aus dem Leben der Bienen* (Bd. 1). Springer Berlin Heidelberg.
<https://doi.org/10.1007/978-3-662-00696-2>
- Van Nest, B. N., & Moore, D. (2012). Energetically optimal foraging strategy is emergent property of time-keeping behavior in honey bees. *Behavioral Ecology*, 23(3), 649–658. <https://doi.org/10.1093/beheco/ars010>
- Wehner, R., & Gehring, W. J. (2013). *Zoologie*. Georg Thieme Verlag.